متن ارائه‌ی آزمایش ۶ — طراحی انکوباتور

حدود ۶ دقیقه | پوریا عرفانزاده، فاطیما تیمارچی، آیه صابری

0:00 • شروع

سلام. من پوریا عرفانزاده هستم، با فاطیما تیمارچی و آیه صابری. آزمایش ششم ما طراحی واحد کنترل یک انکوباتور بود. تو این چند دقیقه می‌خوام بگم صورت مسئله چی بود، چه تصمیم‌های طراحی‌ای گرفتیم، چطور تستش کردیم، و نتیجه‌ی سنتز چی شد.

0:20 • صورت مسئله

یه محفظه داریم با یه سنسور دما، یه هیتر، و یه کولر که پنکه داره. سنسور دمای بین منفی ۱۰ تا مثبت ۶۰ درجه رو می‌خونه و به صورت یه عدد هشت‌بیتی علامت‌دار می‌ده به کنترلر. کنترلر باید دو تا تصمیم بگیره: اوّل اینکه هیتر و کولر کِی روشن و خاموش بشن، دوم اینکه اگه کولر روشنه، دور پنکه چقدر باشه. دستور کار این رفتار رو با دو تا state diagram داده، برای همین طرح ما هم طبیعتاً دو تا FSM شد، هر دو هم Moore.

0:50 • دو ماشین حالت

FSM اوّل سه تا حالت داره. S1 حالت idle که هر دو خاموشن. دما بره بالای ۳۵، می‌ره S2 که کولر روشنه و تا دما نیاد زیر ۲۵ برنمی‌گرده. از S1 دما بیاد زیر ۱۵، می‌ره S3 که هیتر روشنه و تا نره بالای ۳۰ برنمی‌گرده.

FSM دوم دور پنکه رو تنظیم می‌کنه: سه تا حالت کاری با دورهای ۴ و ۶ و ۸ دور بر ثانیه، به‌علاوه‌ی یه حالت OUT که یعنی غیرفعّال. بالای ۴۰ می‌ره روی ۶، بالای ۴۵ می‌ره روی ۸.

قید مهم، پیوند این دوتاست: FSM دوم فقط وقتی از OUT میاد بیرون که FSM اوّل تو

S2 باشه، یعنی کولر روشن باشه. منطقیه، وقتی کولر خاموشه تنظیم دورش معنی نداره. ما هم به جای اینکه دوباره عددهای ۳۵ و ۲۵ رو تکرار کنیم، مستقیم به حالت FSM اوّل نگاه کردیم؛ این‌جوری کلّی‌تره.

1:40 • تصمیم‌های طراحی

سه تا تصمیم طراحی رو می‌خوام برجسته کنم.

اوّل، همه‌ی نامساوی‌ها اکیدن. یعنی دمای دقیقاً ۳۵ گذار نمی‌سازه؛ از ۳۶ به بعد کولر روشن می‌شه. این رو جداگانه هم تست کردیم.

دوم، آستانه‌های رفت و برگشت با هم فرق دارن و همین hysteresis می‌سازه. کولر بالای ۳۵ روشن می‌شه ولی تا زیر ۲۵ خاموش نمی‌شه. این عمدیه و جلوی chattering رو می‌گیره، یعنی اینکه هیتر و کولر دمِ یه آستانه هی روشن و خاموش بشن. یه نتیجه‌ی جالبش اینه که اگه سیستم داره گرم می‌کنه، تو ۲۰ درجه هم هیتر هنوز روشنه.

سوم، سلامت سنسور. دستور کار نگفته بود اگه سنسور عدد بیرون از بازه بده چی کار کنیم، ولی چون ورودی علامت‌داره می‌تونه بده. ما یه سیگنال validity ساختیم و یه خروجی Error_Sensor و یه حالت fail-safe اضافه کردیم. نکته‌ی ظریفش اینه که validity مستقیم روی منطق خروجی اثر می‌ذاره، نه فقط روی گذار حالت؛ یعنی به‌محض دیدن مقدار نامعتبر هیتر و کولر بلافاصله خاموش می‌شن، بدون اینکه منتظر لبه‌ی کلاک بمونن. برای یه کنترلر ایمنی‌محور این رفتار درستیه.

یه نکته‌ی ریز هم بگم: برای فعّال کردن FSM پنکه از next_state ماشین اوّل استفاده کردیم نه current_state. اگه حالت جاری رو ملاک می‌ذاشتیم، دقیقاً تو همون لبه‌ای که کولر روشن می‌شه پنکه هنوز تو OUT بود و برای یه پالس کلاک، کولر روشن ولی دورش صفر گزارش می‌شد.

اینجا می‌خوام یه‌کم بیشتر وقت بذارم، چون به نظرم جالب‌ترین بخش کار بود.

سؤال اصلی این بود که یه واحد کنترل رو اصلاً چطور باید تست کرد. اگه فقط بشینیم دستی چند تا دما بدیم و ببینیم خروجی چی می‌شه، داریم مدار رو تو یه حلقه‌ی *باز* تست می‌کنیم؛ چون دمایی که می‌دیم هیچ ربطی به کاری که خود کنترلر داره می‌کنه نداره. تو دنیای واقعی که این‌طور نیست: کنترلر هیتر رو روشن می‌کنه، دما بالا می‌ره، و همون دمای بالا رفته دوباره می‌شه ورودی خودش. برای همین دو تا تست‌بنچ نوشتیم که دو تا سؤال کاملاً متفاوت رو جواب می‌دن.

تست‌بنچ اوّل — شبیه‌سازی حلقه‌بسته. اینجا داخل خودِ تست‌بنچ یه «انکوباتور قلّابی» نوشتیم؛ بهش می‌گن plant model یعنی مدل فرایند. این سخت‌افزار نیست و سنتز هم نمی‌شه، فقط کد شبیه‌سازیه. کارش اینه که هر ۱۰۰ نانوثانیه — که ما قرارداد کردیم معادل «یه دقیقه» باشه — نگاه می‌کنه ببینه کنترلر الان چه فرمانی داده، و بر اساسش دمای محفظه رو عوض می‌کنه.

فرمولش خیلی ساده‌ست. تغییر دما تو هر دقیقه، جمع سه تا سهمه: یه بار گرمایی بیرونی که خودمون تعیین می‌کنیم، به‌علاوه‌ی ۳ درجه اگه هیتر روشن باشه، منهای سهم کولر که به دور پنکه بستگی داره — دور ۴ یعنی منفی ۱، دور ۶ یعنی منفی ۲، و دور ۸ یعنی منفی ۶. آخرش هم دما رو بین منفی ۱۰ و مثبت ۶۰ محدود می‌کنیم که از بازه‌ی معتبر سنسور بیرون نزنه.

نکته‌ی اصلی اینجاست: خروجی همین مدل مستقیم وصله به ورودی temperature کنترلر. یعنی حلقه بسته شد — کنترلر روی دما اثر می‌ذاره و دما دوباره ورودی کنترلر می‌شه. پس ما دیگه دما رو دیکته نمی‌کنیم؛ فقط شرط اوّلیّه و بار بیرونی رو می‌ذاریم و تماشا می‌کنیم سیستم چی کار می‌کنه.

سناریویی که ساختیم اینه: از ۱۰ درجه شروع می‌کنیم که سرده، پس کنترلر باید خودش هیتر رو روشن کنه. هفت دقیقه گرم می‌شه تا برسه به ۳۱ که هیتر خاموش می‌شه. بعد یه بار گرمایی ۴ درجه بر دقیقه اعمال می‌کنیم، دما می‌ره بالا، کولر روشن می‌شه و دور پنکه پلّه‌پلّه می‌ره ۴ و ۶ و ۸. بعد بار رو برمی‌داریم، دما میاد پایین، دور برمی‌گرده ۶ و بعد ۴، و آخرش زیر ۲۵ کولر کامل خاموش می‌شه.

این‌ها رو هم دستی نگاه نمی‌کنیم؛ خود تست‌بنچ با wait منتظر هر اتّفاق می‌مونه و وقتی افتاد PASS چاپ می‌کنه. یه watchdog هم گذاشتیم: اگه گذاری که منتظرشیم اصلاً هیچ‌وقت رخ نده، شبیه‌سازی به‌جای اینکه تا ابد معلّق بمونه، با یه پیام روشن تموم می‌شه.

آخر این تست‌بنچ fault injection هم داریم. یه mux گذاشتیم که وقتی کلید خطا فعّال بشه، به‌جای دمای مدل یه عدد نامعتبر بره روی ورودی کنترلر —— یه بار مثبت ۷۰ و یه بار منفی ۲۰. هر دو بار چک می‌کنیم که هم خروجی‌ها بلافاصله امن بشن، و هم بعد از چند لبه ماشین حالت واقعاً بره تو حالت SAFE.

ولی این تست‌بنچ یه ضعف مهم داره. مسیر دما اینجا طبیعیه، و همین باعث می‌شه از هر آستانه فقط از یه سمت رد بشیم و هیچ‌وقت هم دقیقاً روی خودِ عدد آستانه نایستیم. مثلاً وقتی دما داره ۴ درجه در دقیقه بالا می‌ره، از ۳۵ می‌پره به ۳۹؛ پس اصلاً تست نشد که تو دمای دقیقاً ۳۵ چی می‌شه. و دقیقاً همین‌جاست که فرق $<$ و $\leq$ خودش رو نشون می‌ده.

تست‌بنچ دوم —— آزمون هدفمند مرزها. پس تست‌بنچ دوم رو نوشتیم که درست برعکس اوّلیه: هیچ مدلی نداره، دما رو مستقیم و پلّه‌ای اعمال می‌کنیم، و بعد از هر پلّه هر چهار تا خروجی رو با مقدار انتظاری مقایسه می‌کنیم.

قاعده‌ش اینه که برای هر آستانه دقیقاً دو تا مقدار می‌دیم: خودِ عدد آستانه که نباید هیچ گذاری بسازه، و اوّلین مقدار بعدش که باید بسازه. مثلاً ۳۵ می‌دیم و انتظار داریم کولر خاموش بمونه، بعد ۳۶ می‌دیم و انتظار داریم کولر روشن بشه و دور بره روی ۴. همین

کار رو برای ۴۰ و ۴۱، برای ۴۵ و ۴۶، برای ۲۵ و ۲۴، برای ۱۵ و ۱۴، و برای ۳۰ و ۳۱ تکرار می‌کنیم. مرزهای اعتبار سنسور رو هم همین‌طور: ۶۰ باید معتبر باشه ولی ۶۱ خطا بده، منفی ۱۰ معتبر باشه ولی منفی ۱۱ خطا.

یه نکته‌ی ریز هم داشت که ارزش گفتن داره: بعد از هر پلّه سه تا لبه‌ی کلاک صبر می‌کنیم، نه یکی. دلیلش اینه که یه پرش بزرگ دما می‌تونه بیشتر از یه گذار پشت‌سرهم تو ماشین پنکه بسازه —— مثلاً از دور ۸ مستقیم بیاد روی ۴ که خودش دو تا گذاره —— و خروجی رو باید بعد از اینکه هر دو ماشین آروم گرفتن سنجید، نه وسط راه.

در مجموع ۲۵ تا وارسی شد و همه‌شون pass شدن.

جمع‌بندی این بخش: تست‌بنچ اوّل به این سؤال جواب می‌ده که «آیا این سیستم تو یه سناریوی واقع‌نما درست کار می‌کنه و به وضعیّت مطلوب می‌رسه؟» و تست‌بنچ دوم به این سؤال که «آیا تک‌تک آستانه‌ها دقیقاً همونی‌ان که نمودار حالت گفته؟» هیچ‌کدوم جای اون یکی رو نمی‌گیره.

4:55 • سنتز و رمزگذاری حالت

بعدش رفتیم Quartus. طرح خیلی کوچیکه، ۱۷ تا پین و زیر یک درصد ظرفیّت قطعه.

چون کلّ طرح از دو تا FSM ساخته شده، فرصت خوبی بود که اثر state encoding رو اندازه بگیریم. یه بار با binary کامپایل کردیم، یه بار با one-hot؛ کد کاملاً یکسان، فقط یه setting فرق داره.

نتیجه جالب بود: binary هم منابع کمتری گرفت، ۴۰ در برابر ۴۳ عنصر منطقی و ۵ در برابر ۸ رجیستر، و هم Fmax بالاتری داد، ۴۲۰ در برابر ۳۳۶ مگاهرتز. این خلاف انتظار متعارفه که می‌گه one-hot به قیمت مساحت سریع‌تره. دلیلش اینه که critical path این طرح اصلاً decode کردن حالت نیست، زنجیره‌ی مقایسه‌کننده‌های هشت‌بیتی دماست

که قبل از منطق next-state قرار داره. با این تعداد کمِ حالت، decode دوبیتی تقریباً مجّانیه، ولی one-hot رجیستر بیشتری به مسیر اضافه می‌کنه. حرف کلّی‌ش اینه که rule of thumb جای اندازه‌گیری رو نمی‌گیره.

تحلیل زمانی رو هم با قید ۵۰ مگاهرتز و مدل بدترین شرایط زدیم؛ slack حدود ۱۷ نانوثانیه‌ست. البته تو کاربرد واقعی سنسور دقیقه‌ای یه بار نمونه می‌ده، پس اصلاً به این حدود نزدیک نمی‌شیم.

5:45 • جمع‌بندی

آخرش قطعه رو صریح انتخاب کردیم، هر ۱۷ تا پین رو با Pin Planner به پین فیزیکی نسبت دادیم با استاندارد ۲/۵ ولت، دوباره کامپایل کردیم که منابع عوض نشد، و assembler بدون خطا فایل پیکربندی رو ساخت. چون آزمایشگاه مجازی بود روی برد فیزیکی پروگرام نکردیم و ادّعایی هم درباره‌ش نداریم.

جمع‌بندی: دو تا ماشین حالت هم‌کار با یه وابستگی شرطی، دو تا testbench که دو تا سؤال مختلف رو جواب می‌دن، و یه مقایسه‌ی encoding که نشون داد باید اندازه گرفت. ممنون، اگه سؤالی هست در خدمتم.